Python Activity-Based Learning

10 case studies:

1. Progressive electricity Billing

2. Hospital Billing and Patient Classification

3.Employee Payroll and Deduction Analysis

4.Retail Billing with Multiple Discount Rules

5. Multi-Plan Telecommunications Cost Optimization

6.Railway Fare and Reservation Calculator

7. Warehouse Inventory Reconciliation

8. Smart Parking Fee Management

9.Attendance Recovery Computation

10. ATM Transaction Sequence Simulation

1.Progressive electricity billing

Problem Statement: Write a Python program to calculate an electricity bill based on the number of units consumed using progressive billing slabs

Algorithm:

1.Start the program.

2.Read the consumer's name.

3.Read the number of electricity units consumed.

4.Check whether the units are negative.

5.If units are negative, display "Invalid units consumed!"

6.Otherwise, calculate the bill using progressive slabs:

First 100 units: ₹2 per unit.

Next 100 units: ₹3 per unit.

Next 100 units: ₹5 per unit.

Next 100 units: ₹5 per unit.

Above 300 units: ₹7 per unit.

7.Display the consumer's name and units consumed.

8.Display the total electricity bill.

9.Stop the program

Code:

Code:

```
name = input ("Enter consumer name: ")
units = float (input ("Enter units consumed: "))
if units < 0:
        print ("Invalid units consumed!")
else:
        if units <= 100:
            bill = units * 2
        elif units <= 200:
            bill = (100 * 2) + (units - 100) * 3
        elif units <= 300:
            bill = (100 * 2) + (100 * 3) + (units - 200) * 5
        else:
            bill = (100 * 2) + (100 * 3) + (100 * 5) + (units - 300) * 7
print ("Consumer Name:", name)
print ("Units Consumed:", units)
print ("Total Bill: Rs.", round (bill, 2))
```

Expected output:

Enter consumer name: Ravi

Enter units consumed: 250

Consumer Name: Ravi

Units Consumed: 250.0

Total Bill: Rs. 750.0

Actual output:

Enter consumer name: Ravi

Enter units consumed: 250

Consumer Name: Ravi

Units Consumed: 250.0

Total Bill: Rs. 750.0

Output screenshot:

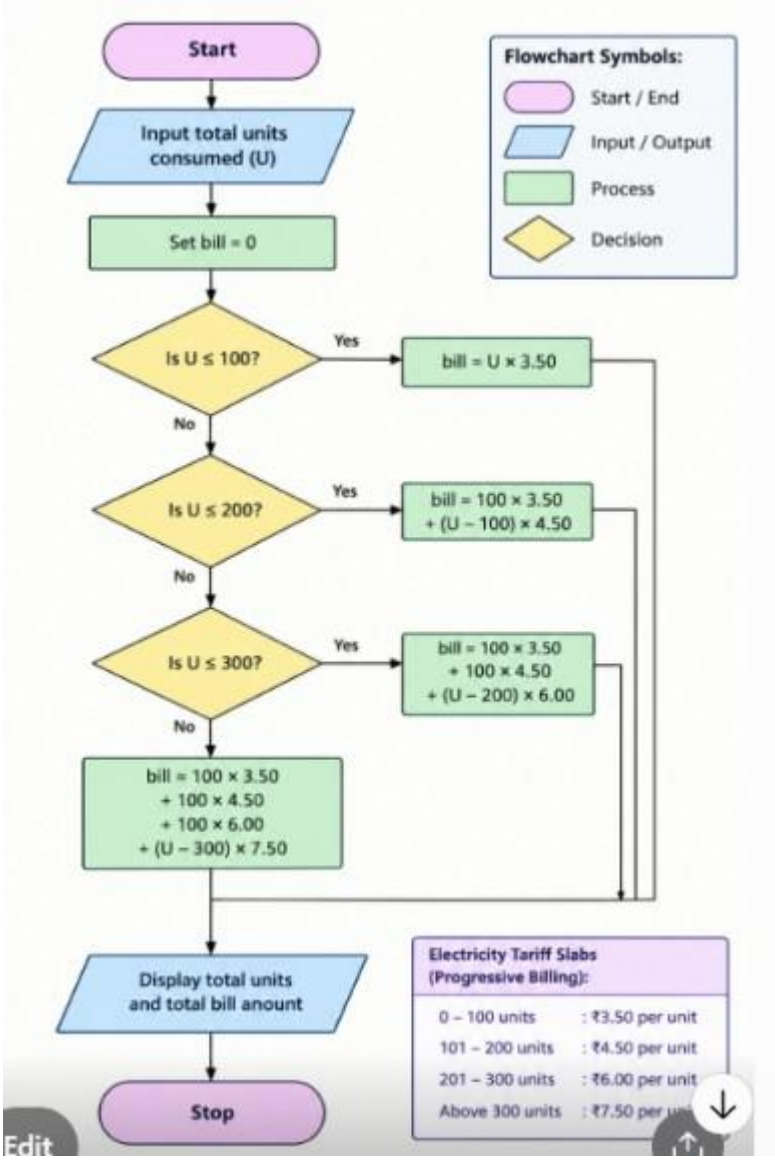
```python
name = input("Enter consumer name: ")
units = float(input("Enter units consumed: "))
if units < 0:
    print("Invalid units consumed!")
else:
    if units <= 100:
        bill = units * 2
    elif units<= 200:
        bill = (100 * 2) + (units - 100) * 3
    elif units <= 300:
        bill = (100 * 2) + (100 * 3) + (units - 200) * 5
    else:
        bill = (100 * 2) + (100 * 3) + (100 * 5) + (units - 300) * 7
print("Consumer Name:", name)
print("Units Consumed:", units)
print("Total Bill: Rs.", round(bill, 2))
```

```
Run  python21

C:\Users\karun\OneDrive\Desktop\python\.venv\Scripts\python.exe "C:\Us
Enter consumer name: hari
Enter units consumed: 214
Consumer Name: hari
Units Consumed: 214.0
Total Bill: Rs. 570.0

Process finished with exit code 0
```

Flowchart:



Assumptions:

• The billing rates are fixed for this assignment: n2 for the first 100 units, n3 for the next 100 units, and n5 for
units above 200.

• The bill is calculated using progressive slabs; each rate applies only to the units in that slab.

• Zero units is valid and results in a bill of n0.

• The number of units can be entered as an integer or decimal number.

• Taxes, fixed charges, subsidies, late fees, and other utility-specific charges are not included.

• The program assumes that the user enters a numeric value for units; handling non-numeric text is outside the basic version .

Objectives:

• To accept the consumer's name and electricity units as input.

• To reject negative unit values and display an appropriate error message.

• To apply the correct rate to each progressive billing slab.

• To calculate and display the final electricity bill clearly.
• To practise decision-making in Python using if, elif, and else statements.

---

2. Hospital Billing and Patient Classification

Problem statement: Hospitals need a consistent way to classify patients for a basic administrative workflow and calculate an itemized bill from service fees, room charges, consultation, laboratory and medicine charges. Manual calculations can lead to arithmetic mistakes, inconsistent classifications and unclear bills. This project demonstrates a small Pythonconsole application that accepts patient details, classifies the patient by age group and care type, calculates
charges using defined sample rates, and prints a bill.

Algorithm:
1. Start
2. Read patient ID, name, age and patient type.
3. If the patient is an Inpatient, read the number of admitted days; otherwise use zero room days.
4. Read consultation, laboratory and medicine charges.
5. Validate age, patient type, admission days and charge values.
6. Assign age group: Child, Adult or Senior; append the care type to form the classification.
7. Choose the service fee based on patient type. Calculate room charges for Inpatients only.
8. Calculate subtotal = service fee + room charges + consultation + laboratory + medicines.
9. Calculate tax = subtotal × 5%; calculate total = subtotal + tax.
10. Print the patient classification and itemized bill . stop

Code:

```
name = input("Enter patient name: ")

age = int(input("Enter age: "))

if age <= 12:

   category = "Child"

elif age >= 60:

   category = "Senior"

else:

   category = "Adult"
```

```
print("1. Inpatient")
print("2. Outpatient")
choice = int(input("Enter choice: "))
if choice == 1:
    days = int(input("Enter admission days: "))
    bill = 1500 + days * 2500
else:
    bill = 300
consult = float(input("Consultation fee: "))
lab = float(input("Lab charges: "))
medicine = float(input("Medicine charges: "))
total = bill + consult + lab + medicine
total = total * 1.05
print("\n--- Hospital Bill ---")
print("Patient:", name)
print("Category:", category)
print("Total Bill: Rs.", round(total, 2))
```

expected output:

Enter patient name: hari

Enter age: 19

1. Inpatient

2. Outpatient

Enter choice: 2

Consultation fee: 330

Lab charges: 230

Medicine charges: 660

--- Hospital Bill ---

Patient: hari

Category: Adult

Total Bill: Rs. 1596.0

Actual output:

Enter patient name: hari

Enter age: 19

1. Inpatient

2. Outpatient

Enter choice: 2

Consultation fee: 330

Lab charges: 230

Medicine charges: 660

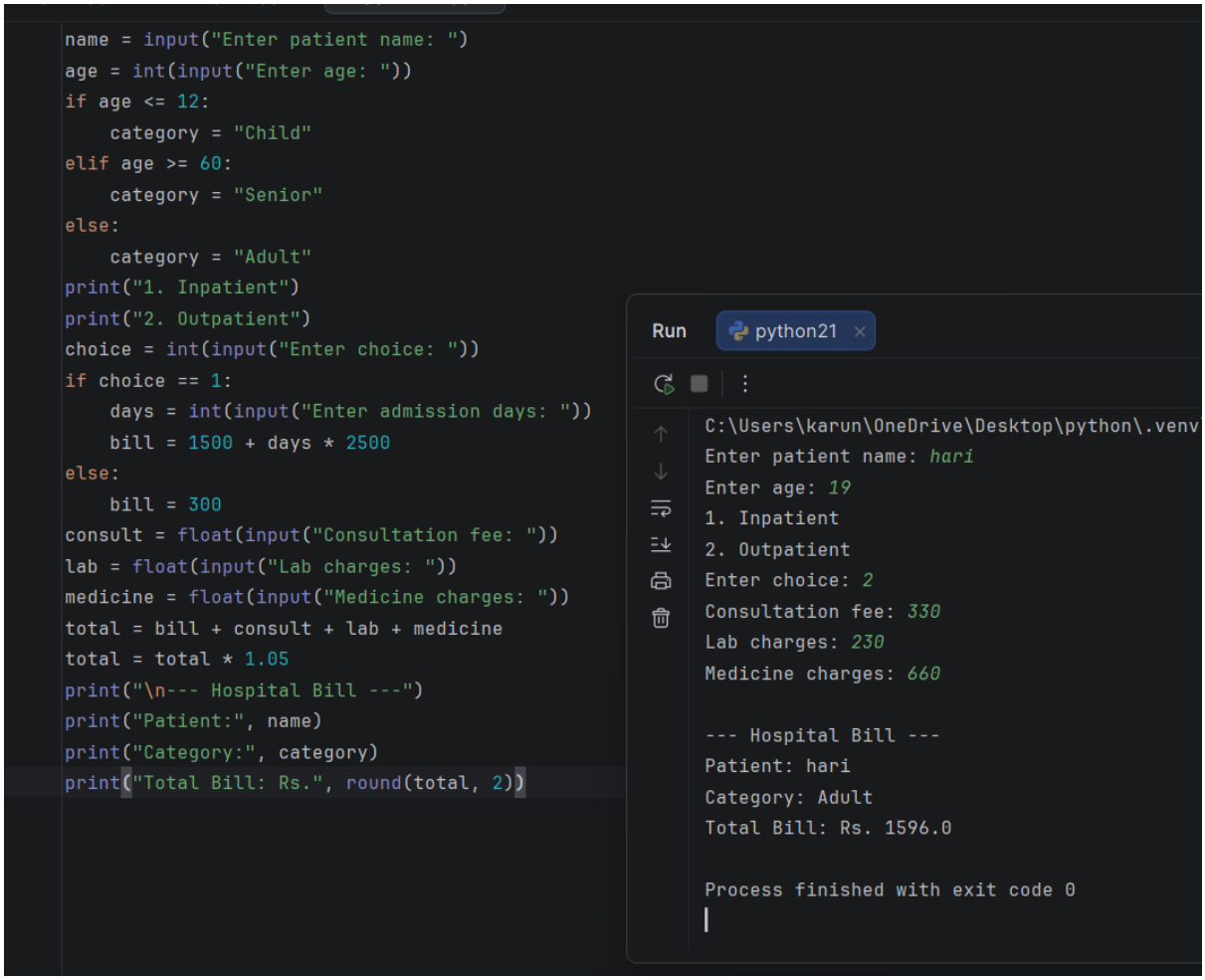
--- Hospital Bill ---

Patient: hari

Category: Adult

Total Bill: Rs. 1596.0

Output screenshot:



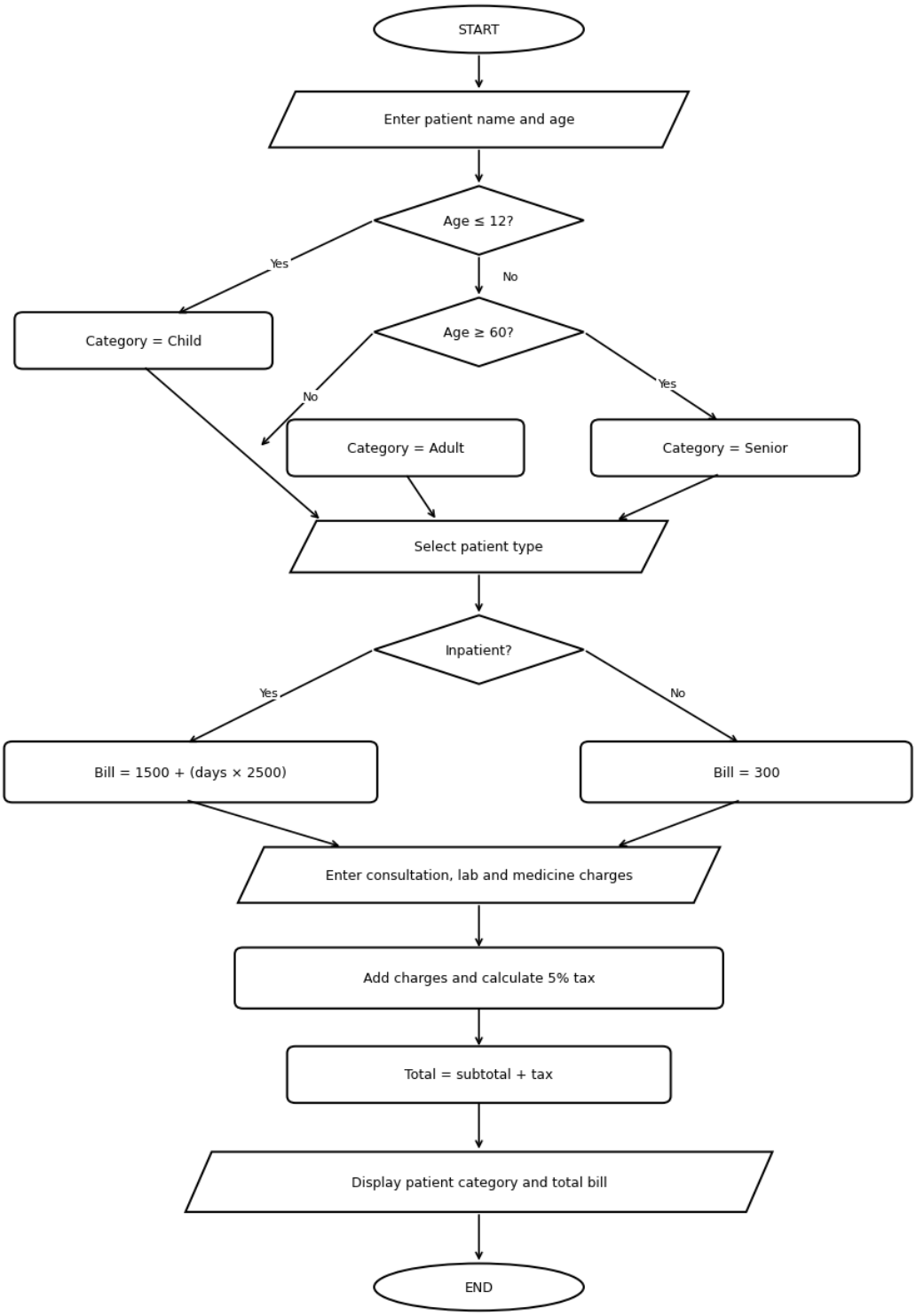
```
name = input("Enter patient name: ")
age = int(input("Enter age: "))
if age <= 12:
    category = "Child"
elif age >= 60:
    category = "Senior"
else:
    category = "Adult"
print("1. Inpatient")
print("2. Outpatient")
choice = int(input("Enter choice: "))
if choice == 1:
    days = int(input("Enter admission days: "))
    bill = 1500 + days * 2500
else:
    bill = 300
consult = float(input("Consultation fee: "))
lab = float(input("Lab charges: "))
medicine = float(input("Medicine charges: "))
total = bill + consult + lab + medicine
total = total * 1.05
print("\n--- Hospital Bill ---")
print("Patient:", name)
print("Category:", category)
print("Total Bill: Rs.", round(total, 2))
```

```
Run  python21
C:\Users\karun\OneDrive\Desktop\python\.venv
Enter patient name: hari
Enter age: 19
1. Inpatient
2. Outpatient
Enter choice: 2
Consultation fee: 330
Lab charges: 230
Medicine charges: 660

--- Hospital Bill ---
Patient: hari
Category: Adult
Total Bill: Rs. 1596.0

Process finished with exit code 0
```

Flowchart:


START
Enter patient name and age
Age ≤ 12?
Yes
No
Category = Child
Age ≥ 60?
No
Yes
Category = Adult
Category = Senior
Select patient type
Inpatient?
Yes
No
Bill = 1500 + (days × 2500)
Bill = 300
Enter consultation, lab and medicine charges
Add charges and calculate 5% tax
Total = subtotal + tax
Display patient category and total bill
END

Assumption:

1.Patients are classified as Child (0–12 years), Adult (13–59 years), and Senior (60 years and above).

2. There are two patient types: Inpatient and Outpatient.

3.Inpatient service fee is ₹1,500.

4. Outpatient service fee is ₹300.

5. Room charges are ₹2,500 per day for inpatients only.

6. Consultation, laboratory, and medicine charges are entered by the user.

7.A sample tax rate of 5% is applied to the total charges before tax.

8. All charges and admission days must be non-negative.

9. Patient details are entered manually through the keyboard.

10. The program calculates and displays the patient's classification and final hospital bill.

Objective:

1.To develop a simple Python program for hospital billing.

2.To classify patients into Child, Adult, and Senior categories based on age.

3.To identify patients as Inpatients or Outpatients.

4.To calculate hospital charges, including room, consultation, laboratory, and medicine fees.

5.To calculate the final bill by adding the applicable tax.

6.To display patient details, classification, and total billing amount.

7.To reduce manual calculation errors and improve billing accuracy.

3.Employee Payroll and Deduction Analysis

Problem statement: Employee payoff and deduction analysis is an important process in payroll management. Organizations need to calculate employees' gross salaries, allowances, overtime payments, bonuses, and other earnings while accurately accounting for deductions such as income tax, Provident Fund (PF), health insurance, unpaid leave, and loan repayments.

Algorithm:

1. Start the program.
2. Enter the employee's basic salary.
3. Enter allowances, overtime pay, and bonus amounts.
4. Enter deductions such as tax, PF, insurance, and loan repayment.
5. Check whether all entered amounts are valid and non-negative.
6. Calculate the gross salary by adding basic salary, allowances, overtime, and bonus.
7. Calculate the total deductions by adding all deduction amounts.
8. Calculate the net salary by subtracting total deductions from gross salary.
9. Display the employee's gross salary, total deductions, and net salary.
10. If the deductions are greater than the gross salary, display a warning.
11. Stop the program.

Code:

```
basic = float(input("Enter basic salary: "))
allowances = float(input("Enter allowances: "))
deductions = float(input("Enter total deductions: "))
gross = basic + allowances
netsalary = gross - deductions
print("Gross Salary:", gross)
print("Total Deductions:", deductions)
print("Net Salary:", netsalary)
```

expected output:

Enter basic salary: 20000

Enter allowances: 2000

Enter total deductions: 1000

Gross Salary: 22000.0

Total Deductions: 1000.0

Net Salary: 21000.0

Actual output:

Enter basic salary: 20000

Enter allowances: 2000

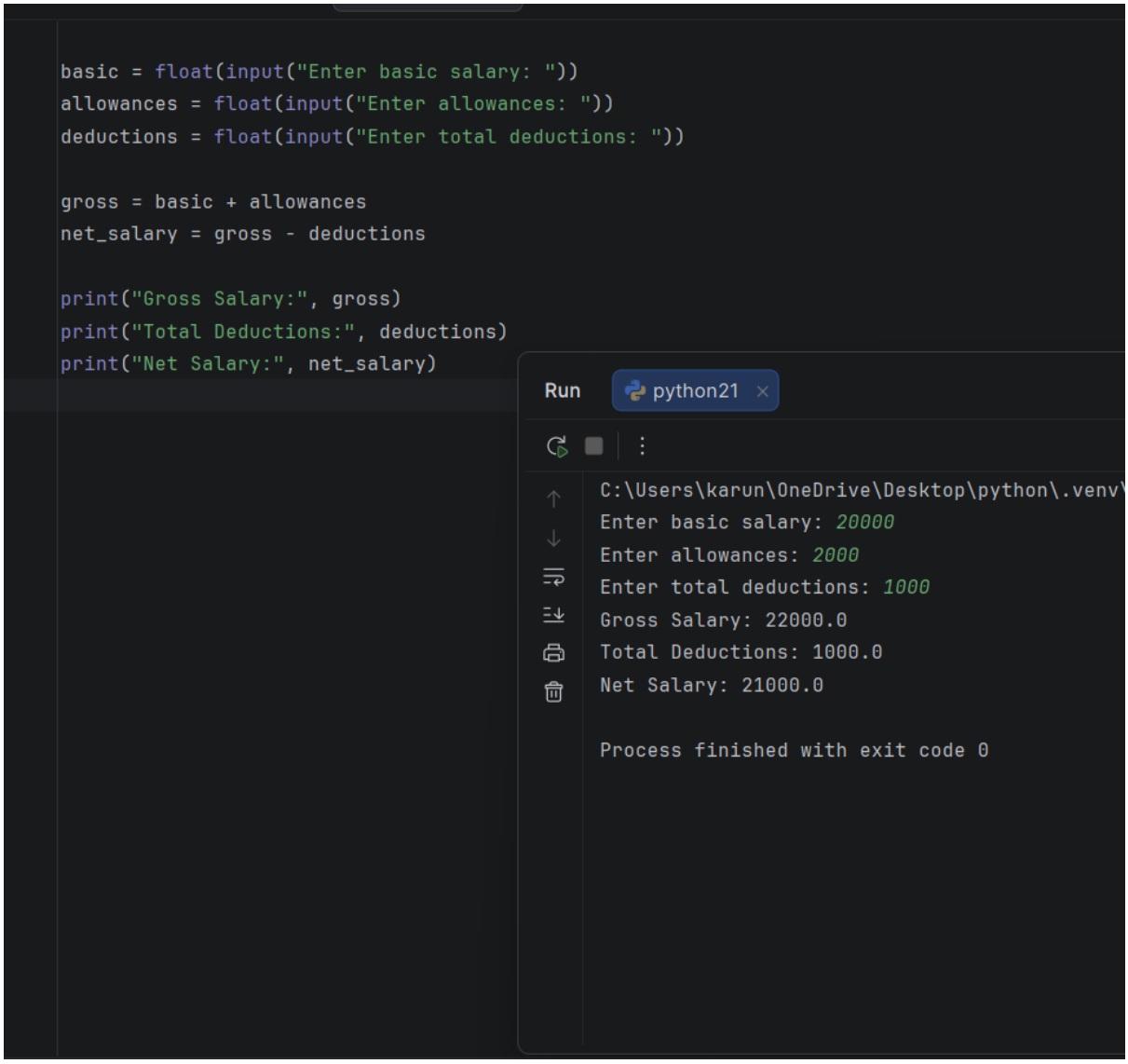
Enter total deductions: 1000

Gross Salary: 22000.0

Total Deductions: 1000.0

Net Salary: 21000.0

Output screenshot:



Assumption:

1. Each employee has a defined basic salary for the relevant pay period.

2.Allowances, overtime payments, bonuses, and other earnings are provided as input.

3.All monetary values are represented in Indian rupees (₹).

4.Tax, PF, insurance, and other deduction amounts are supplied or calculated using configured rules.

5. The applicable deduction rates and salary policies are assumed to be predefined and approved by the organization.

6.Unpaid leave deductions are calculated using the salary divisor specified by the organization's payroll policy.

7.Loan and salary advance recoveries cannot exceed the outstanding balance.

Objective:

1.To calculate employees' gross salary based on basic pay, allowances, overtime, and bonuses.

2. To calculate employee deductions, including income tax, PF, insurance, unpaid leave, and loan repayments.

3.To determine the net salary or take-home pay after subtracting total deductions from gross earnings.

4. To automate payroll calculations using Python programming.

5.To validate employee salary inputs and identify invalid or inconsistent values.

---

4. Retail Billing with Multiple Discount Rules

Problem statement: Retail shops often calculate discounts manually, which can cause mistakes. This program calculates the bill amount and
applies a discount rule automatically.

Algorithm:

1.Start the program.

2.Enter the bill amount.

3.If the bill is ₹5,000 or more, apply a 10% discount.

4.Otherwise, if the bill is ₹2,000 or more, apply a 5% discount.

5.Otherwise, apply no discount.

6.Calculate the final bill by subtracting the discount.

7.Display the discount and final bill.

8.Stop the program.

Code:

```
bill = float(input("Enter bill amount: "))
if bill >= 5000:
   discount = bill * 0.10
elif bill >= 2000:
```

```
    discount = bill * 0.05
else:
    discount = 0
final_bill = bill - discount
print("Discount:", discount)
print("Final Bill:", final_bill)
```

expected output:

Enter bill amount: 4000

Discount: 200.0

Final Bill: 3800.0

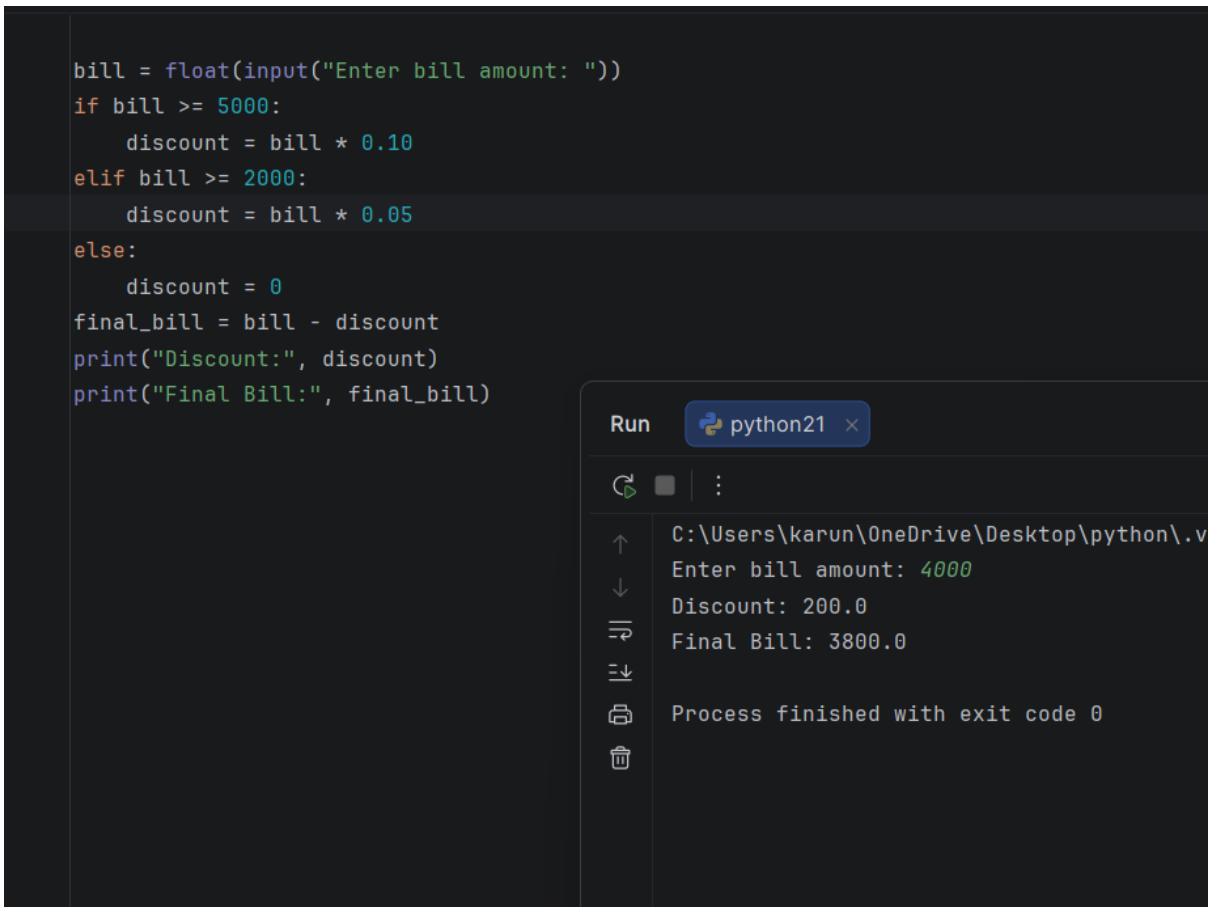
Actual output:

Enter bill amount: 4000

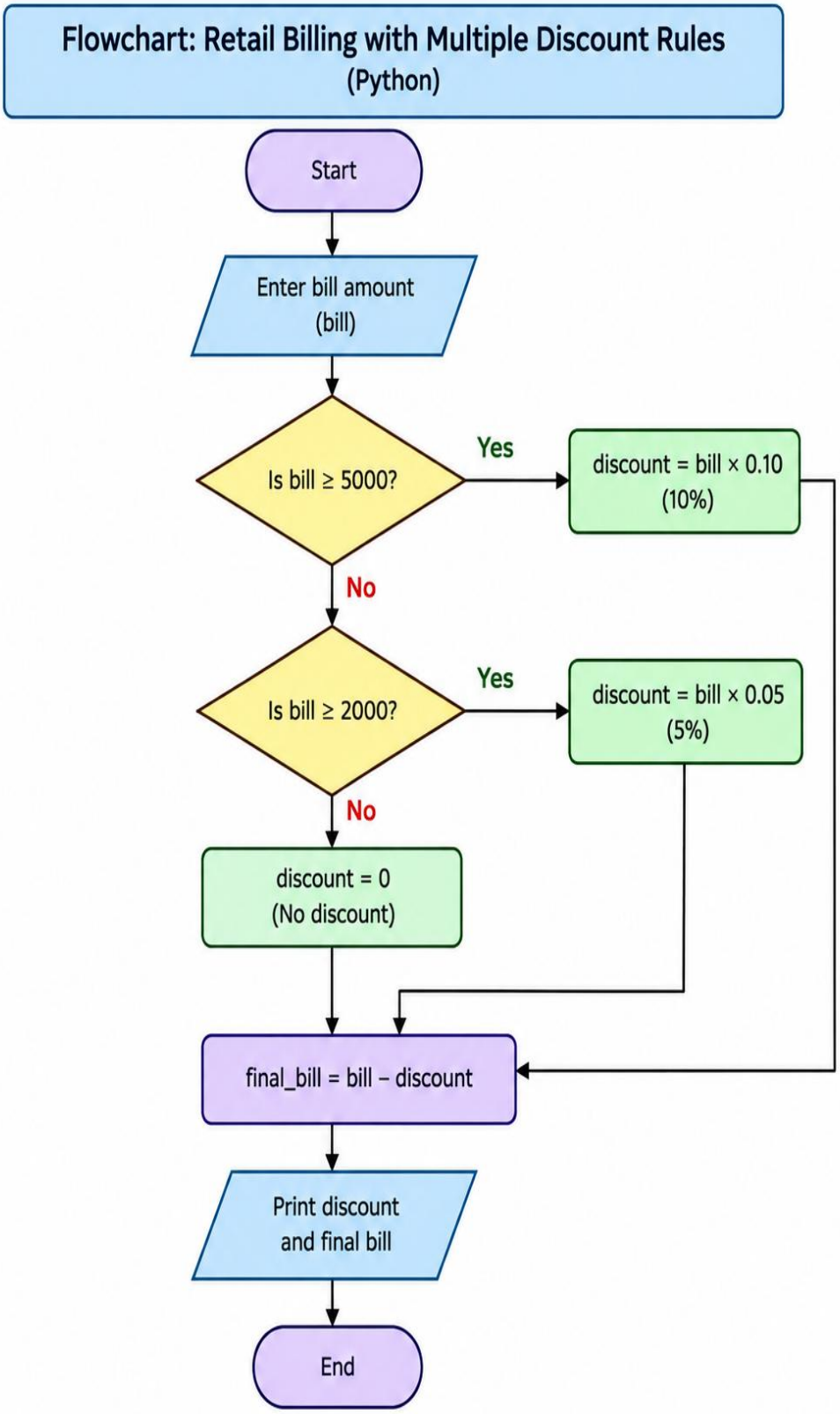
Discount: 200.0

Final Bill: 3800.0

Output screenshot:



Flowchart:


Flowchart: Retail Billing with Multiple Discount Rules
(Python)
Start
Enter bill amount
(bill)
Is bill ≥ 5000?
Yes
discount = bill × 0.10
(10%)
No
Is bill ≥ 2000?
Yes
discount = bill × 0.05
(5%)
No
discount = 0
(No discount)
final_bill = bill – discount
Print discount
and final bill
End

Assumption:

1. The customer enters a valid bill amount.
2. If the bill is ₹5,000 or more, a 10% discount is applied.
3. If the bill is ₹2,000 or more but less than ₹5,000, a 5% discount is applied.
4. If the bill is below ₹2,000, no discount is applied.
5. Only one discount rule is applied to each bill.
6. The final bill is calculated by subtracting the discount from the original bill.
7. Negative bill amounts are considered invalid.

Objectives:

• Calculate the total bill.
• Apply a discount based on the bill amount.
• Display the discount and final amount.

3. Assumptions

• The user enters a valid purchase amount.
• If the bill is n5,000 or more, the discount is 10%.
• If the bill is n2,000 or more but less than n5,000, the discount is 5%.
• Bills below n2,000 receive no discount

5.Multi-plan telecommunication cost optimization

Problem statement: People use different phone plans with different monthly prices and data limits. Choosing a suitable plan can be confusing.This Python program compares plan costs and identifies the cheapest plan that meets the user's data requirement.

Algorithm:

1.Start the program.

2.Enter the required data in GB.

3.Store the available plans and their prices.

4.Select plans that provide enough data.

5.Find the cheapest suitable plan.

6.Display the plan name and monthly cost.

7.If no plan matches, display “No suitable plan found.”

Code:

```
plans = [("Basic", 10, 199),
      ("Standard", 25, 299),
      ("Premium", 50, 399)]
data = int(input("Enter required data (GB): "))
suitable = [p for p in plans if p[1] >= data]
if suitable:
  best = min(suitable, key=lambda p: p[2])
  print("Best Plan:", best[0])
  print("Monthly Cost: Rs.", best[2])
else:
  print("No suitable plan found")
```

expected output:

Enter required data (GB): 20

Best Plan: Standard

Monthly Cost: Rs. 299

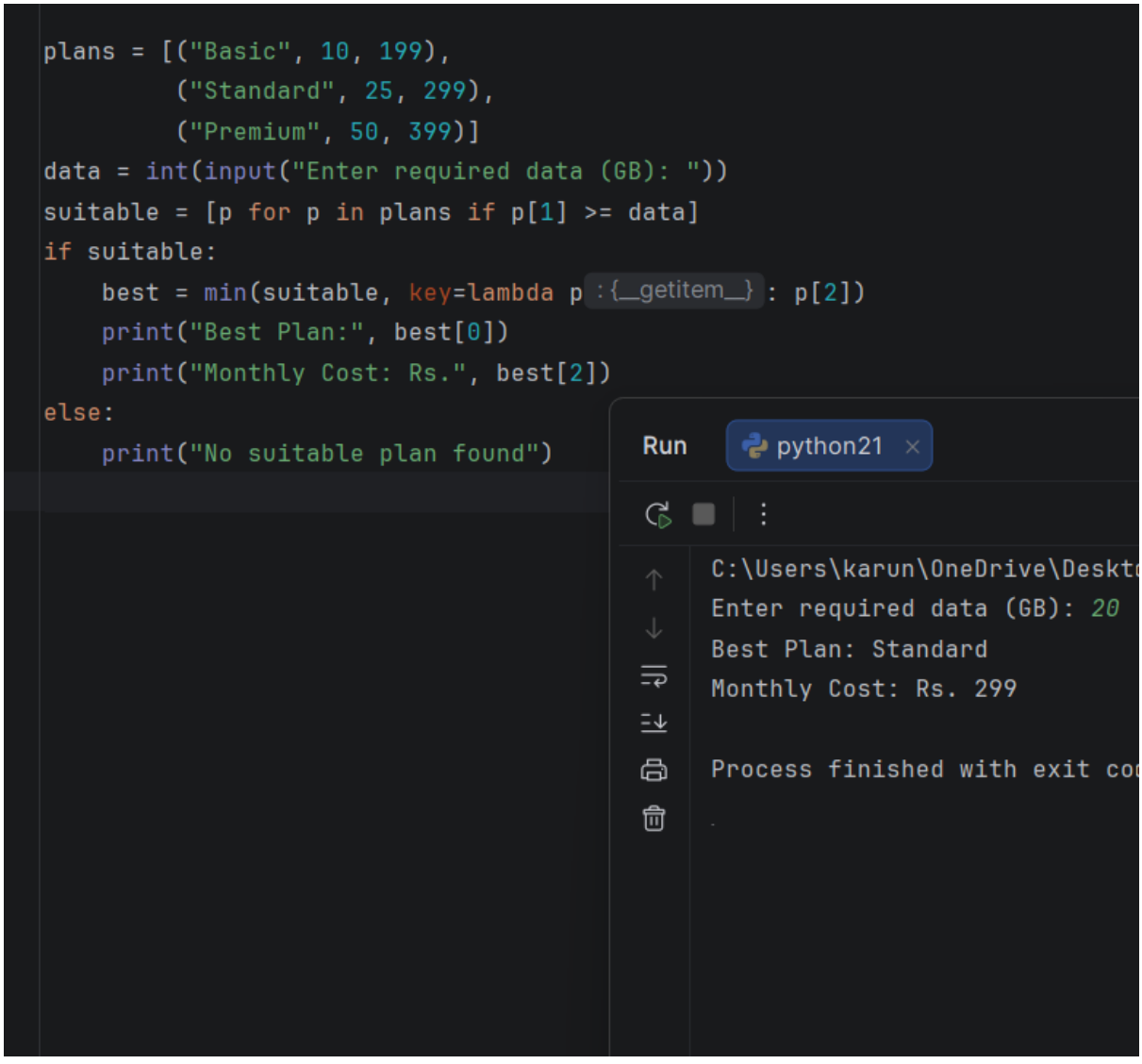
Actual output:

Enter required data (GB): 20

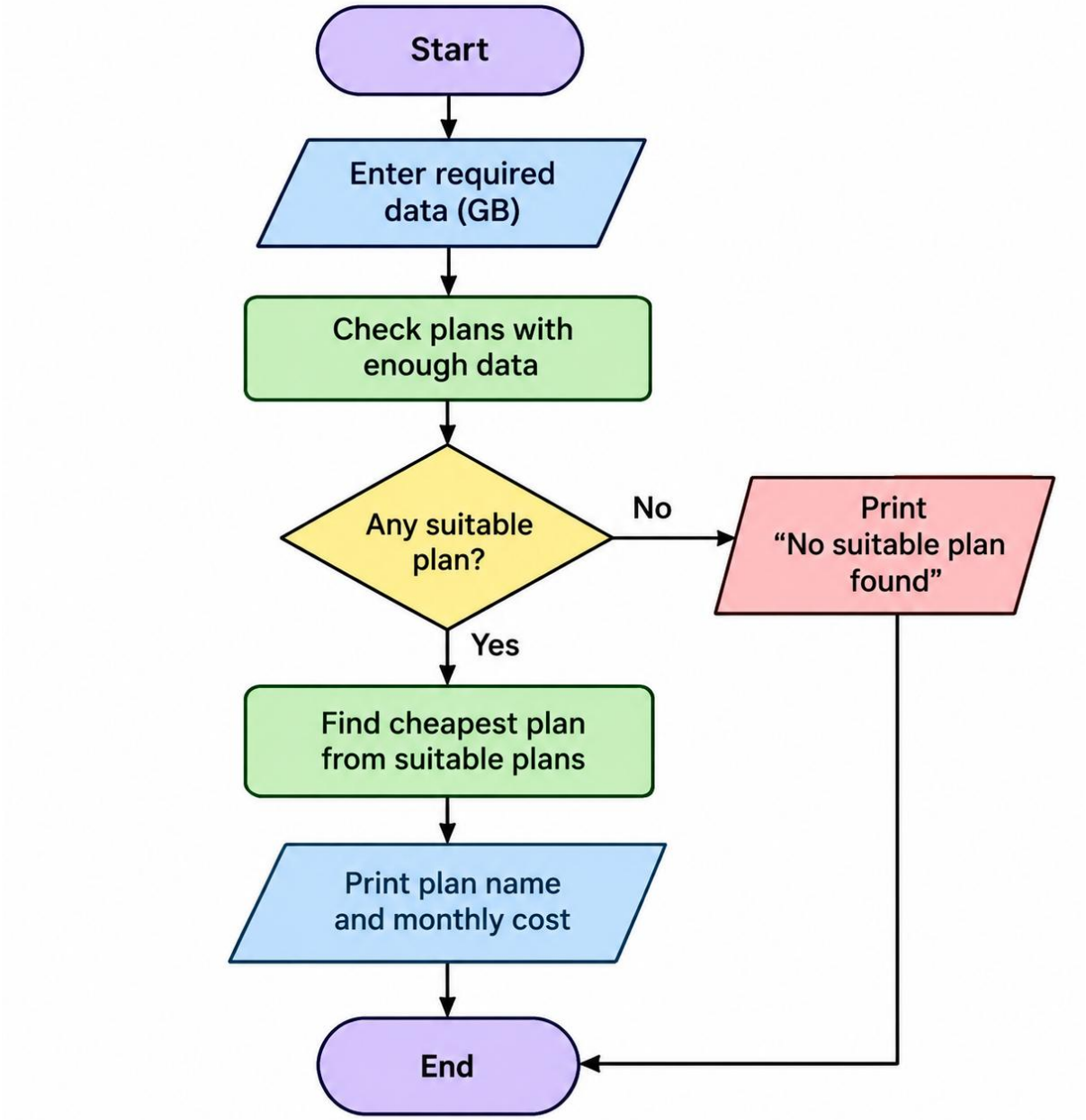
Best Plan: Standard

Monthly Cost: Rs. 299

Output screenshot:



Flowchart:



Objectives:

1.To compare different telecommunications plans.

2.To find the cheapest plan based on required data.

3.To display the selected plan and its monthly cost.

4.To reduce telecommunications expenses.

Assumptions:

1.Plan prices and data limits are predefined.

2.The user enters the required data in GB.

3.Only plans that provide sufficient data are considered.

4.The plan with the lowest price is selected.

5.Call charges, SMS, taxes, and network coverage are not considered.

6.Railway Fare and Reservation Calculator

Problem statement : Passengers need to estimate train ticket costs based on travel class and number of passengers. This simple Python program
calculates the total fare and displays a basic reservation summary.

Algorithm:

1.Start the program.

2. Enter passenger name and number of passengers.

3. Select the travel class.

4. Get the fare for the selected class.

5. Calculate total fare = Fare × Number of passengers.

6.Display passenger details and total fare.

7.If the input is invalid, display an error message.

8.Stop the program.

Code:

```
fares = {"Sleeper": 200, "AC": 500, "First Class": 1000}
name = input("Enter passenger name: ")
```

```
n = int(input("Enter number of passengers: "))
travel_class = input("Enter class (Sleeper/AC/First Class): ")
if n > 0 and travel_class in fares:
   total = fares[travel_class] * n
   print("Passenger:", name)
   print("Total Fare: Rs.", total)
else:
   print("Invalid input")
```

expected output:

Enter passenger name: hari

Enter number of passengers: 10

Enter class (Sleeper/AC/First Class): sleeper

Invalid input

Actual output:

Enter passenger name: hari

Enter number of passengers: 10

Enter class (Sleeper/AC/First Class): sleeper

Invalid input

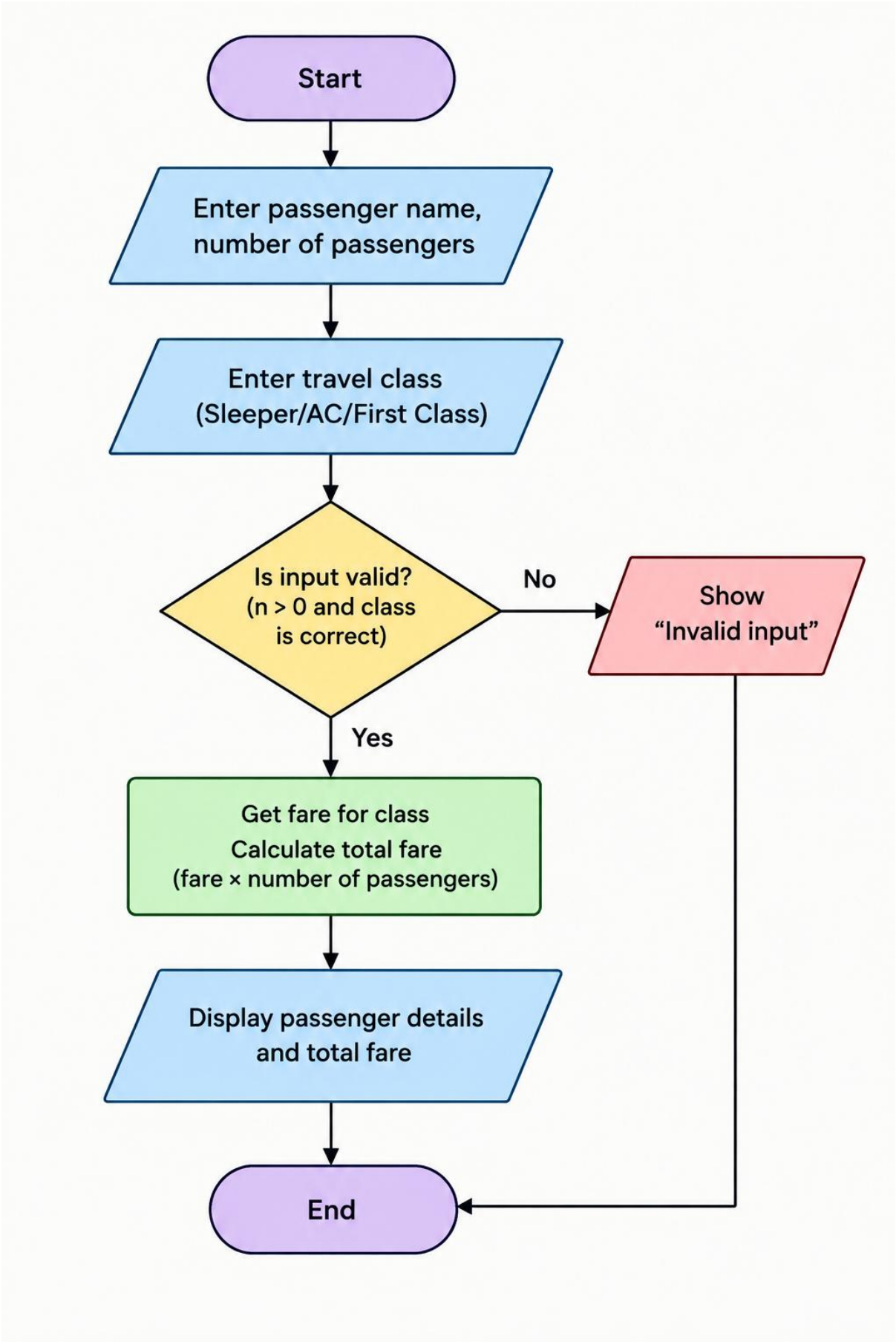
Output screenshot:

```python
fares = {"Sleeper": 200, "AC": 500, "First Class": 1000}
name = input("Enter passenger name: ")
n = int(input("Enter number of passengers: "))
travel_class = input("Enter class (Sleeper/AC/First Class): ")
if n > 0 and travel_class in fares:
    total = fares[travel_class] * n
    print("Passenger:", name)
    print("Total Fare: Rs.", total)
else:
    print("Invalid input")
```

```
Run  python21
C:\Users\karun\OneDrive\Desktop\python\.venv\Scripts
Enter passenger name: hari
Enter number of passengers: 10
Enter class (Sleeper/AC/First Class): sleeper
Invalid input

Process finished with exit code 0
```

Flowchart:


Start
Enter passenger name, number of passengers
Enter travel class (Sleeper/AC/First Class)
Is input valid? (n > 0 and class is correct)
No
Show "Invalid input"
Yes
Get fare for class
Calculate total fare (fare × number of passengers)
Display passenger details and total fare
End

Objectives:

1.To calculate railway ticket fares.

2.To select a travel class.

3.To calculate the total fare based on the number of passengers.

4.To display passenger details and total fare.

Assumptions:

1.Ticket fares are predefined for each travel class.

2.The passenger enters a valid name and number of passengers.

3.All passengers travel in the same class.

4.Sample fares are used for calculation.

5.Actual seat availability and ticket booking are not included.

---

7. Warehouse Inventory Reconciliation

Problem statement: Warehouse staff compare recorded stock with the actual quantity counted. This program checks both quantities and reports whether stock matches, is missing, or is extra**.**

Algorithm:

1.Start the program.

2.Enter the item name, recorded quantity, and actual quantity.

3.Calculate the difference between actual and recorded stock.

4.If the difference is zero, display “Stock matches.”

5.If positive, display “Extra stock.”

6.If negative, display “Missing stock.”

7.Stop the program.

Code:

```
recorded = int(input("Enter recorded stock: "))
actual = int(input("Enter actual stock: "))
difference = actual - recorded
if difference == 0:
   print("Stock matches")
elif difference > 0:
   print("Extra stock:", difference)
else:
   print("Missing stock:", abs(difference))
```

expected output:

Enter recorded stock: 200

Enter actual stock: 500

Extra stock: 300

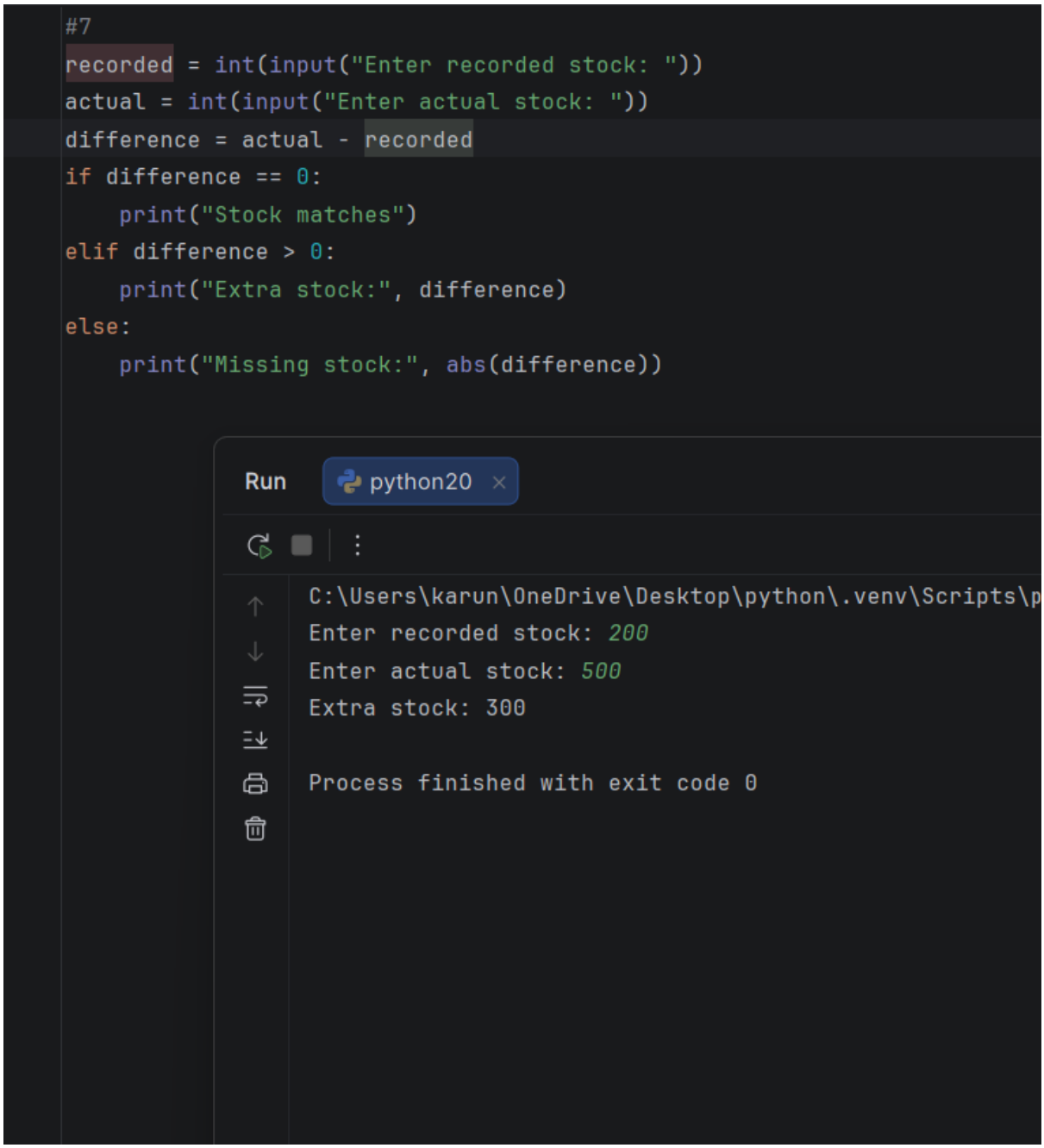
Actual output:

Enter recorded stock: 200

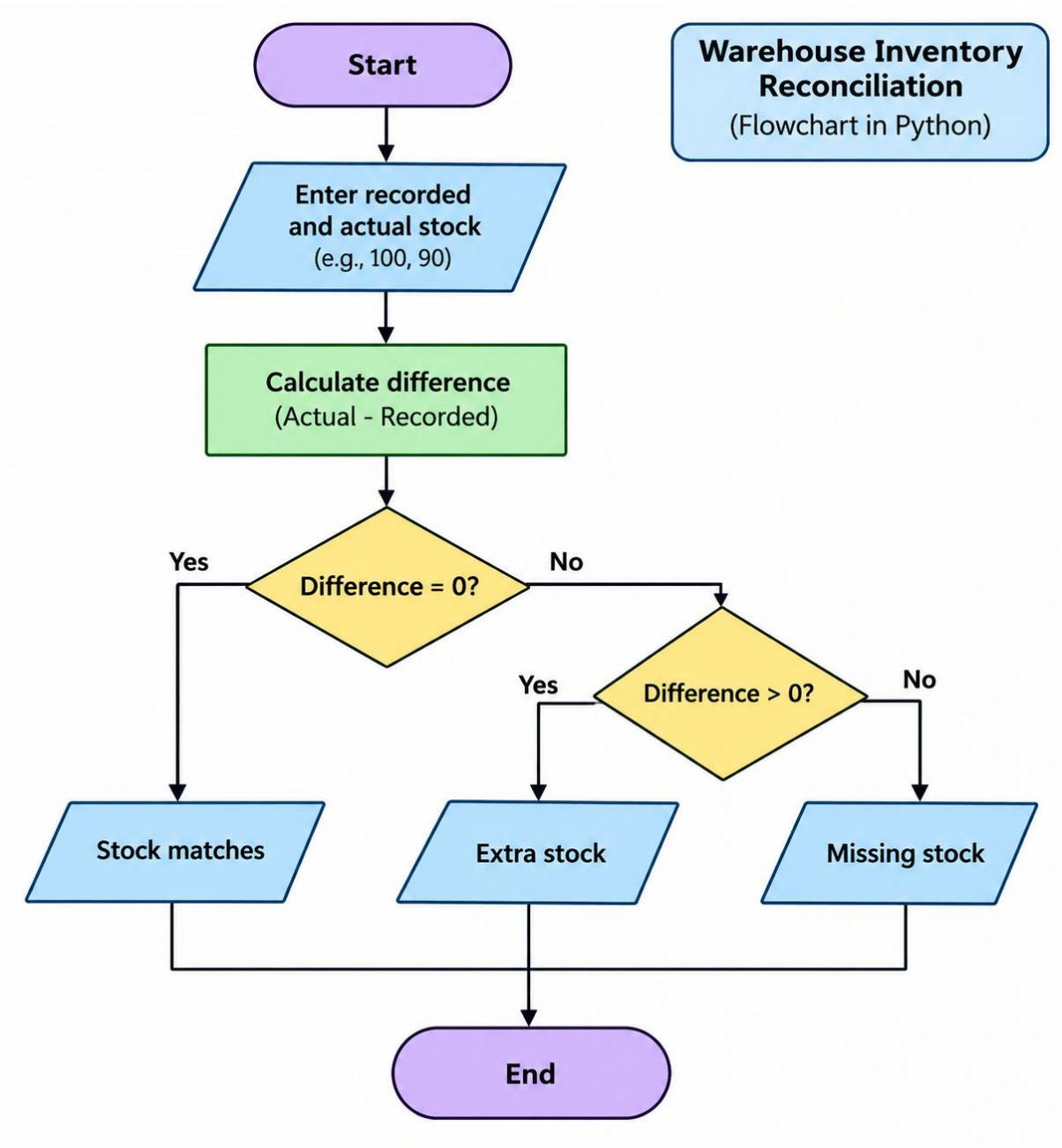
Enter actual stock: 500

Extra stock: 300

Output screenshot:



Flowchart:


Warehouse Inventory Reconciliation
(Flowchart in Python)
Start
Enter recorded and actual stock
(e.g., 100, 90)
Calculate difference
(Actual - Recorded)
Difference = 0?
Yes
No
Difference > 0?
Yes
No
Stock matches
Extra stock
Missing stock
End

Objectives:

1.To compare recorded stock with actual stock.

2.To identify extra or missing stock.

3 .To display the stock difference using Python.

2. Assumptions

1.Recorded and actual stock quantities are entered as whole numbers.

2.Stock quantities cannot be negative.

3.The entered values are correct.

4.The program compares one item at a time.

9.smart packing fee management

Problem statement: The Smart Parking and Fee Management system calculates parking fees based on vehicle type and parking duration. It accepts the vehicle type and number of hours parked, calculates the fee, and displays the amount to be paid. This reduces manual calculations and makes parking management easier.

Algorithm:

1.Start.

2.Enter vehicle type and parking hours.

3.Set the rate: Car – ₹30/hour, Bike – ₹15/hour.

4.Calculate fee = Rate × Hours.

5.Display the parking fee.

6.Stop.

Code:

```
vehicle = input("Enter vehicle (car/bike): ").lower()
```

```
hours = int(input("Enter parking hours: "))
if vehicle == "car":
    fee = 30 * hours
elif vehicle == "bike":
    fee = 15 * hours
else:
    fee = -1
if fee >= 0 and hours > 0:
    print("Parking fee: Rs.", fee)
else:
    print("Invalid input")
```

expected output:

Enter vehicle (car/bike): bike

Enter parking hours: 2

Parking fee: Rs. 30

Actual output:

Enter vehicle (car/bike): bike

Enter parking hours: 2

Parking fee: Rs. 30

Output screenshot:

```python
#9
vehicle = input("Enter vehicle (car/bike): ").lower()
hours = int(input("Enter parking hours: "))
if vehicle == "car":
    fee = 30 * hours
elif vehicle == "bike":
    fee = 15 * hours
else:
    fee = -1
if fee >= 0 and hours > 0:
    print("Parking fee: Rs.", fee)
else:
    print("Invalid input")
```

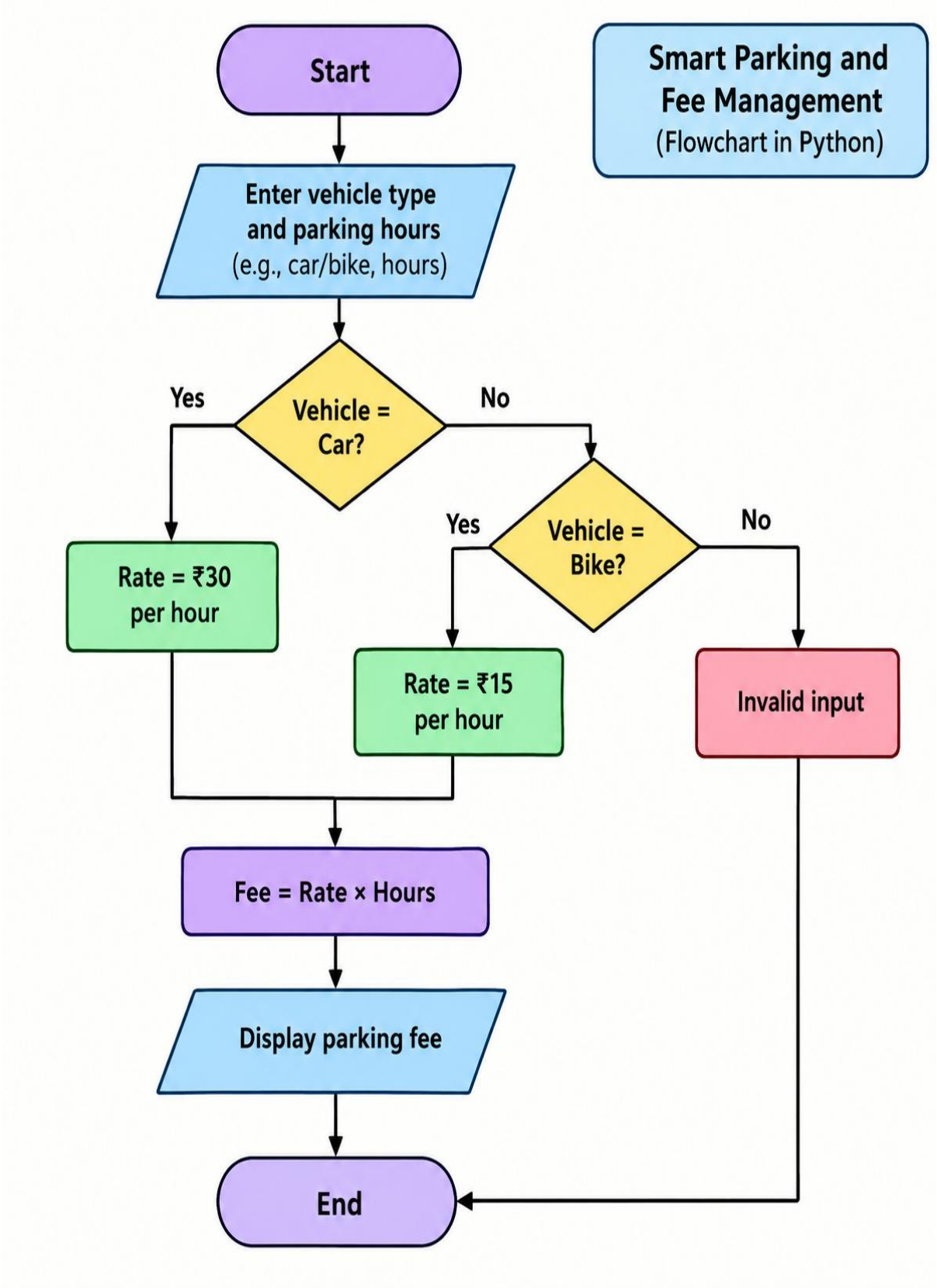
Run python19

```
C:\Users\karun\OneDrive\Desktop\python\.venv\Scripts\pyt
Enter vehicle (car/bike): bike
Enter parking hours: 2
Parking fee: Rs. 30
```

This view is read-only

```
Process finished with exit code 0
```

Flowchart:


Smart Parking and Fee Management
(Flowchart in Python)
Start
Enter vehicle type and parking hours
(e.g., car/bike, hours)
Vehicle = Car?
Yes
No
Rate = ₹30 per hour
Vehicle = Bike?
Yes
No
Rate = ₹15 per hour
Invalid input
Fee = Rate × Hours
Display parking fee
End

Objectives:

1.To calculate parking fees automatically.

2.To calculate fees based on vehicle type and parking hours.

3.To display the total parking fee.

Assumptions:

1.The vehicle type is either a car or a bike.

2.Car parking costs ₹30 per hour.

3.Bike parking costs ₹15 per hour.

4.Parking hours are entered as a positive whole number.

5.The fee is calculated as Rate × Parking Hours.

10.ATM transaction sequence simulation

Problem statement: Create a Python program that simulates a basic ATM transaction. The user enters a PIN, selects a transaction, and the program displays a balance or processes a cash withdrawal.

Algorithm:

1.Start.

2.Enter the PIN.

3.If the PIN is incorrect, display an error.

4.Select balance inquiry or cash withdrawal.

5.Check the balance or enter the withdrawal amount.

6.Validate the amount and update the balance.

7.Display the result.

8.Stop.

Code:

```
balance = 5000
pin = input("Enter PIN: ")
if pin == "1234":
  print("1. Check Balance")
  print("2. Withdraw Cash")
  choice = input("Enter choice: ")
  if choice == "1":
    print("Balance:", balance)
  elif choice == "2":
    amount = int(input("Enter amount: "))
    if 0 < amount <= balance:
      balance -= amount
      print("Remaining balance:", balance)
    else:
      print("Insufficient balance or invalid amount")
  else:
    print("Invalid choice")
else:
  print("Invalid PIN")
```

expected output :

Enter PIN: 1234

1. Check Balance

2. Withdraw Cash

Enter choice: 1

Balance: 5000

Actual output:

Enter PIN: 1234

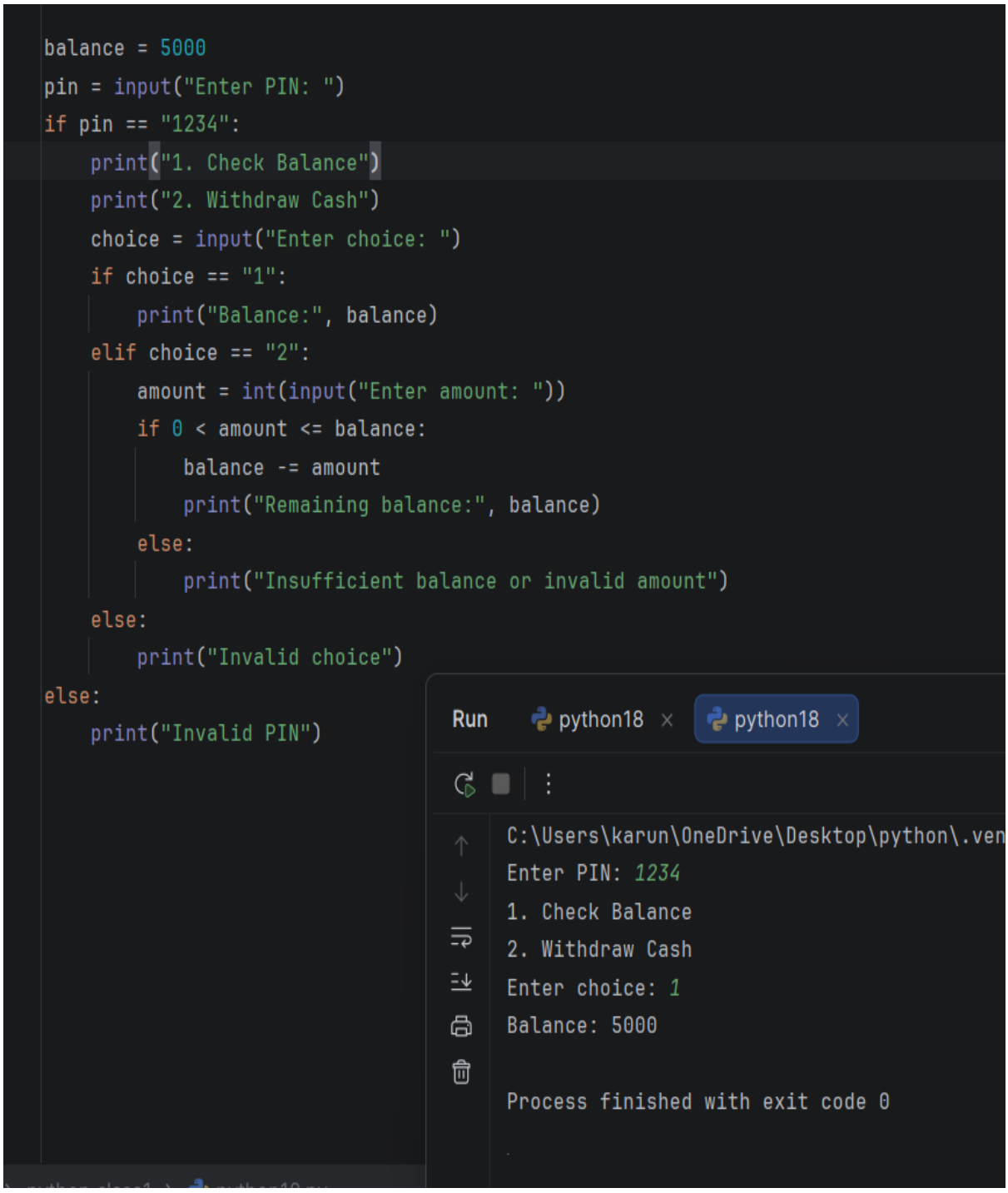
1. Check Balance

2. Withdraw Cash

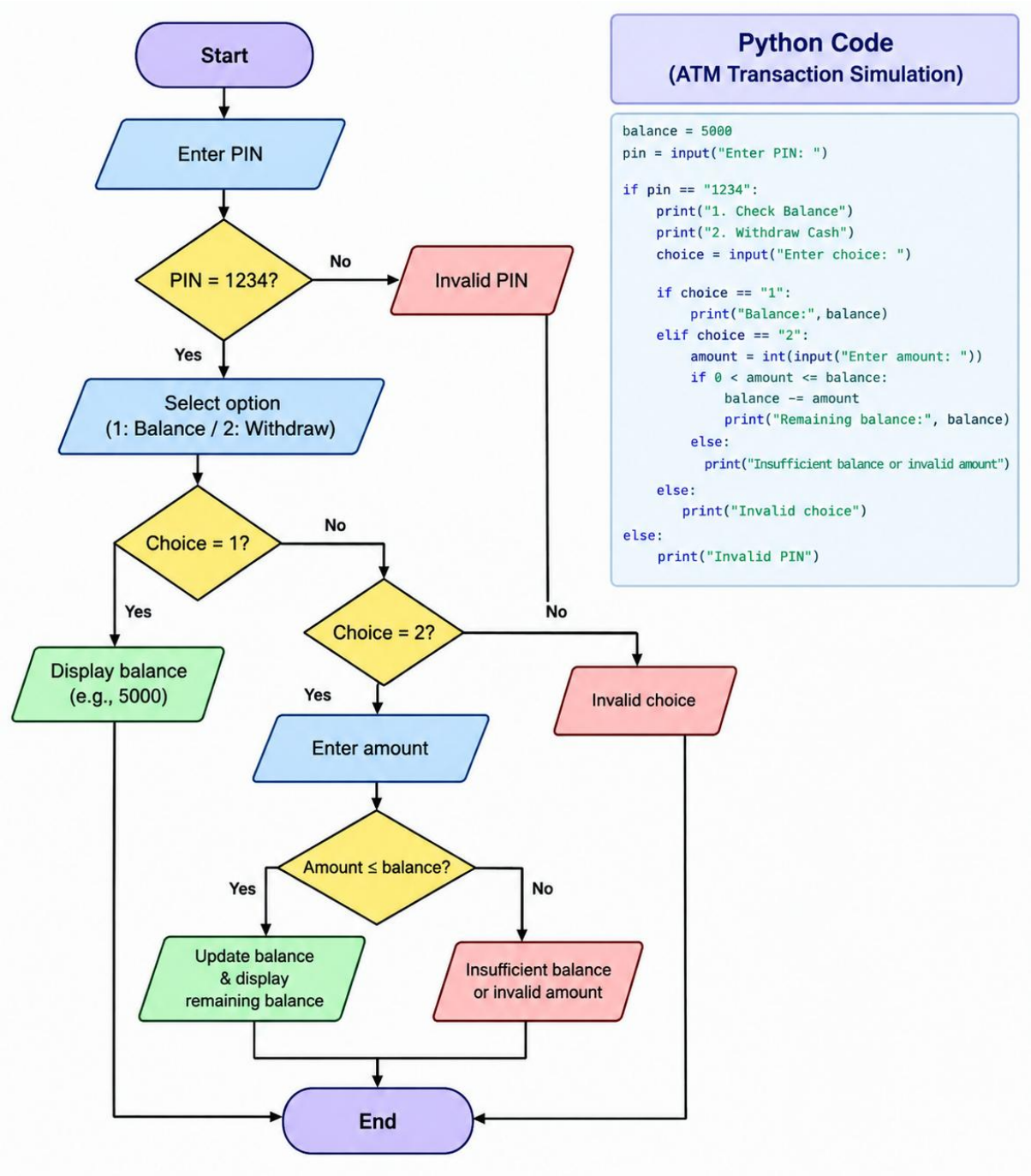
Enter choice: 1

Balance: 5000

Output screenshot:



Flowchart:


Start
Enter PIN
PIN = 1234?
No
Invalid PIN
Yes
Select option
(1: Balance / 2: Withdraw)
Choice = 1?
No
Yes
Display balance
(e.g., 5000)
Choice = 2?
No
Invalid choice
Yes
Enter amount
Amount ≤ balance?
Yes
No
Update balance
& display
remaining balance
Insufficient balance
or invalid amount
End
Python Code
(ATM Transaction Simulation)
balance = 5000
pin = input("Enter PIN: ")
if pin == "1234":
print("1. Check Balance")
print("2. Withdraw Cash")
choice = input("Enter choice: ")
if choice == "1":
print("Balance:", balance)
elif choice == "2":
amount = int(input("Enter amount: "))
if 0 < amount <= balance:
balance -= amount
print("Remaining balance:", balance)
else:
print("Insufficient balance or invalid amount")
else:
print("Invalid choice")
else:
print("Invalid PIN")

Objectives:

1.To simulate basic ATM transactions using Python.

2.To verify the user's PIN.

3.To check account balance and withdraw cash.

4.To display the remaining balance.

Assumptions:

1.The sample PIN is 1234.

2.The initial balance is ₹5,000.

3.Users can check their balance or withdraw cash.

4.The withdrawal amount must be positive and within the available balance.

5.This is a simple simulation, not a real ATM system.